

Neuro-DT Notebook Run Guide

Brain Digital Twin — Alzheimer's Disease Progression Simulation

Seif Hendawy · Arab Academy for Science, Technology and Maritime Transport

Supervisors: Prof. Fahima Maghraby · Assoc. Prof. Ahmed Salem

This document covers two notebooks:

- (1) **NeuroDT_CPU_Workhorse.ipynb** — Azure Standard_E4ds_v4 cluster (CPU only)
- (2) **NeuroDT_GPU_Lab.ipynb** — University GPU lab (full training + ablation)

Contents

Section	Topic
1	CPU Workhorse — Overview & Run Order
2	CPU Workhorse — Ablation Study (what you can run now)
3	GPU Lab — Hardware Requirements & Time Estimates
4	GPU Lab — Environment Setup (step by step)
5	GPU Lab — Data Transfer from Azure
6	GPU Lab — Run Order & Training Improvements
7	GPU Lab — Full Ablation Study
8	Comparing CPU vs GPU Results in Your Thesis

1. CPU Workhorse Notebook

File: NeuroDT_CPU_Workhorse.ipynb | Cluster: Azure Standard_E4ds_v4 | 4 cores · 32 GB RAM · 150 GB disk
· CPU only

1.1 Run Order — Every Session

Cell 3 → Cell 3b → Cell 4 → Cell 7 → Cell 8 → Cell 8b → Cell 9 → Cell 10b
Then run any downstream cell (11 onward) directly.

Cell	Purpose	Run?
3	Imports & Azure workspace connection	Every session
3b	Suppress verbose warnings	Every session
4	Load manifest DataFrame	Every session
5	DICOM validation (20 scans)	Done — skip
6	3D volume validation	Done — skip
7	Preprocessing pipelines & dataset classes	Every session
8	MultimodalTransformer model definition	Every session
8b	Ablation variant model definitions (new)	Every session
9	Training config & class weights	Every session
9b	One-time DICOM → tensor cache	Done — skip
10	K-Fold training loop	Done — skip (checkpoints saved)
10b	Reload checkpoints (use instead of 10)	Every session
10c	Monitor running job	Skip
10d	Selective fold retrain	Skip
11	Evaluation: confusion matrix, per-class AUC, SHAP	Once
12	3D Grad-CAM explainability	Once
13	Markov Chain prognostic engine	Once

Cell	Purpose	Run?
14	Digital Twin assembly	Once
15	What-if simulation: APOE4 & drug intervention	Once
15c	Cognitive reserve simulation	Once
15d	Age sensitivity analysis	Once
15e	Population subgroup comparison	Once
15f	Early vs late intervention timing	Once
16	Single patient Digital Twin inference	As needed

■ Cells NOT suitable for CPU — deferred to GPU lab

Cell 10 (full 5-fold retraining) takes approximately 69–129 hours on CPU. Cell A2 (DL ablation variants A2–A6) takes 13–26 hours each on CPU. Both cells are present in the notebook but print a clear warning if no GPU is detected. Skip them on the CPU machine and use Cell 10b to load the existing checkpoint instead.

2. CPU Workhorse — Ablation Study

Run order for the ablation section (after loading model with Cell 10b):

Cell A1_CPU → Cell A3 → Cell A4_CPU

Cell	What it does	Time on CPU	Thesis value
A1_CPU	Part 1: Evaluates the existing <code>best_model_fold4.pth</code> on the Fold 4 validation set — no retraining, ~2 min. Part 2: Trains the Tabular-Only MLP (A1) — 3-layer MLP on 4 clinical features, no DenseNet forward passes at all. ~5 min.	~10 min total	Quantifies MRI contribution: $\Delta \text{AUC} = 0.912 - \text{A1 result}$. Most important ablation finding.
A3	Trains 4 classical ML models on the same Fold 4 split using tabular features only: Logistic Regression, SVM (RBF), Random Forest (200 trees), Gradient Boosting.	~5 min	Proves deep learning outperforms all classical tabular-only approaches. Answers the question: why not just use an SVM?
A4_CPU	Loads <code>ablation_results.json</code> and produces: (1) results table with GPU-pending placeholders, (2) bar chart saved as 300 DPI PNG for thesis inclusion.	~1 min	Presentable interim ablation table to show supervisors immediately, before GPU lab run.
A5	Recovery cell. Reloads <code>ablation_results.json</code> and shows completed variants and remaining ones.	Instant	Safe resumption if kernel dies mid-ablation. Re-run Cell A2 after this to continue.

3. GPU Lab — Hardware Requirements

Resource	Minimum	Recommended	Why this matters
GPU VRAM	12 GB	16–24 GB	DenseNet121 on 128 ³ volumes at batch 16 uses ~11–14 GB VRAM. 12 GB is the minimum; 16+ GB gives headroom for ablation.
System RAM	32 GB	64 GB	DataLoader workers, model state, and tensor cache loading run in parallel. 32 GB is the absolute floor.
Disk (SSD)	50 GB free	100 GB free	Tensor cache ~13 GB + model checkpoints + ablation checkpoints + MLflow logs + figures.
GPU model	RTX 3080 / V100	A10 / RTX 4090 / A100	Any CUDA 11.8+ capable GPU works. V100 and A10 are common in university lab clusters.
CUDA version	11.8+	12.1+	Required for PyTorch 2.1 with AMP (mixed-precision) support.

Time estimates by GPU model

Task	V100 (16 GB)	A10 (24 GB)	RTX 4090 (24 GB)
5-fold training, 20 epochs each	~10–12 hrs	~8–10 hrs	~6–8 hrs
Ablation DL variants A2–A6 (5 runs)	~4–5 hrs	~3–4 hrs	~2–3 hrs
Classical ML baselines B1–B4	~5 min	~5 min	~5 min
Evaluation, Grad-CAM, analysis	~30 min	~20 min	~15 min
Total across two sessions	~15 hrs	~12 hrs	~9 hrs

Session plan: Book the GPU lab for two sessions. Session 1 (~first booking): run Cell 10_GPU for full 5-fold training, then Cells 11–16 for all analysis. Session 2 (~second booking): run Cells A1 → A2 → A3 → A4 for the complete ablation study.

4. GPU Lab — Environment Setup (Step by Step)

Step 1 — Check CUDA version

Open a terminal on the lab machine and run:

```
nvidia-smi
```

Step 2 — Install Miniconda (skip if already installed)

Run in terminal:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh bash
Miniconda3-latest-Linux-x86_64.sh -b -p $HOME/miniconda3 source
$HOME/miniconda3/etc/profile.d/conda.sh
```

Step 3 — Create and activate the environment

Run:

```
conda create -n bdt-env python=3.10 -y conda activate bdt-env
```

Step 4 — Install CUDA-compatible PyTorch

Match the CUDA version from Step 1:

```
# For CUDA 11.8: pip install torch==2.1.0+cu118 torchvision==0.16.0+cu118 \ --extra-index-url
https://download.pytorch.org/whl/cu118 # For CUDA 12.1: pip install torch==2.1.0+cu121
torchvision==0.16.0+cu121 \ --extra-index-url https://download.pytorch.org/whl/cu121
```

Step 5 — Install all remaining dependencies

Run:

```
pip install "numpy<2.0" pandas scikit-learn tqdm matplotlib \ "monai[all]" pydicom nibabel
mlflow shap hmmlearn \ azure-storage-blob azure-identity \ jupyter ipykernel ipywidgets python
-m ipykernel install --user --name bdt-env \ --display-name "Python (BDT)"
```

Step 6 — Verify GPU is accessible

Run this to confirm PyTorch sees the GPU:

```
python -c "import torch; print(torch.cuda.get_device_name(0));
print(torch.cuda.get_device_properties(0).total_memory/1e9, 'GB')"
```

Step 7 — Set the Azure credential environment variable

Run before starting Jupyter (rotate the secret first — see warning box below):

```
export AZURE_CLIENT_SECRET=""
```

Step 8 — Update file paths in Cell 9_GPU

Before running anything, open Cell 9_GPU and change these two lines to match the lab machine:

```
BEST_MODEL_DIR = Path("./checkpoints") # ← folder where you copied checkpoints/ CACHE_DIR =
Path("./tensor_cache") # ← folder where you copied tensor_cache/
```

Step 9 — Start Jupyter

Run:

```
conda activate bdt-env jupyter notebook --no-browser --port=8888 # Copy the URL from the
terminal output and open it in your browser
```

■ Rotate the Azure client secret before using the GPU lab.

The original secret was visible in notebook history. Generate a new one before transferring files:

Azure Portal → Azure Active Directory → App registrations → App ID c638dc4d-96ec-4457-8797-23902283156b
→ Certificates and secrets → New client secret → copy the value

5. GPU Lab — Data Transfer from Azure

What files to copy to the lab machine

File / Folder	Size	Critical?	Purpose
tensor_cache/*.pt (1,549 files)	~13 GB	✓ Critical	Preprocessed 128 ³ MRI volumes. Training uses these directly — no raw DICOM download needed during training.
checkpoints/best_model_fold4.pth	~90 MB	✓ Critical	The existing Fold 4 model. Used by A0 evaluation and by Cell 10b to resume without retraining.
checkpoints/markov_matrices.pkl	<1 MB	✓ Critical	Markov chain transition matrices (P_full, P_apoe4_pos, P_apoe4_neg). Required by Cells 13–16.
checkpoints/auc_results.json	<1 KB	Recommended	CPU baseline AUC values (CN, MCI, Dementia). Used by Cell COMPARE to show the CPU vs GPU comparison.
NeuroDT_GPU_Lab.ipynb	<1 MB	✓ Critical	This notebook file.

The tensor cache is the most important file to transfer. Without it, each training batch would download and preprocess raw DICOM slices from Azure Blob Storage in real time, adding 8–15 seconds per batch and making training approximately 100× slower. The raw DICOMs in Azure are only accessed on-demand during single-patient inference (Cell 16) and the Streamlit dashboard. No bulk DICOM download is needed.

Transfer options — choose the one that fits your setup

Method	Best for	How
SCP from Azure compute	If you have SSH access to the compute instance	<code>scp -r username@:/mnt/batch/.../tensor_cache . /tensor_cache scp -r username@:/mnt/batch/.../checkpoints ./checkpoints</code>
Azure Storage Explorer (GUI)	Easiest option — no command line	Download the free app from azure.microsoft.com/features/storage-explorer , connect to adnihendawy storage account, open adni-data container, download tensor_cache/ and checkpoints/ folders.
azcopy (fastest — parallel)	Large transfers, resumable if interrupted	See azcopy commands below.

1. Install azcopy on the lab machine: `wget https://aka.ms/downloadazcopy-v10-linux -O azcopy.tar.gz tar -xvf azcopy.tar.gz mv azcopy_*/azcopy /usr/local/bin/` # 2. Generate a SAS

```
token: # Azure Portal → adnihendawy storage account → adni-data container # → Shared access  
signature → set expiry date → Generate SAS # Copy the SAS token (starts with ?sv=...) # 3. Copy  
tensor_cache: azcopy copy \ "https://adnihendawy.blob.core.windows.net/adni-data/tensor_cache?"  
\ "./tensor_cache" --recursive # 4. Copy checkpoints: azcopy copy \  
"https://adnihendawy.blob.core.windows.net/adni-data/checkpoints?" \ "./checkpoints" --recursive
```

Azure Blob Storage access during runtime — no download needed

Training uses only the local tensor cache — no Azure connection is needed during Cell 10_GPU. The Azure service principal (set in Step 7) is only used by Cell 16 for single-patient inference and by the Streamlit dashboard. The AZURE_CLIENT_SECRET environment variable handles authentication automatically.

6. GPU Lab — Run Order & Training Improvements

Session 1 — Full training and evaluation

Setup: Cell 3 → 3b → 4 → 7 → 8 → 8b → Cell 9_GPU

Training: Cell 10_GPU (set FAST_PROTO=False before running — takes 8–12 hrs)

After training: Cell COMPARE → Cell 10b → Cell 11 → Cell 12 → Cell 13 → Cell 14 → Cell 15 → 15c → 15d → 15e → 15f → Cell 16

Session 2 — Complete ablation study

Setup: Cell 3 → 3b → 4 → 7 → 8 → 8b → Cell 9_GPU → Cell 10b

Ablation: Cell A1 → Cell A2 → Cell A3 → Cell A4

If kernel dies during A2: run Cell A5 to check progress, then re-run Cell A2 to resume

Cell-by-cell reference

Cell	Purpose	Notes
9_GPU	GPU training configuration	UPDATE PATHS: set BEST_MODEL_DIR and CACHE_DIR to your local folders before running. Cell auto-detects GPU, sets batch=16, AMP=True, workers=4.
10_GPU	Full 5-fold training with OneCycleLR	Set FAST_PROTO=False for thesis results. Saves best_model_fold{N}.pth for each fold. OneCycleLR with 10% warmup fixes the early stopping issue from the CPU run.
COMPA RE	CPU Workhorse vs GPU Lab side-by-side	Copies auc_results.json from checkpoints/ for CPU numbers. Shows fold-by-fold AUC and the cross-validated mean ± std for the thesis.
10b	Load best checkpoint	Run after Cell 10_GPU to load the best fold model for all downstream cells.
11	Evaluation: confusion matrix, per-class AUC, SHAP	Full SHAP on the imaging branch is now possible on GPU (was CPU OOM before).
12	Grad-CAM explainability	Faster on GPU (~2 min per scan vs ~20 min on CPU).
13–16	Markov Chain, Digital Twin, Simulations, Single Patient	Identical logic to the CPU version. No changes needed.

Why OneCycleLR fixes the early stopping problem

The original CPU training used CosineAnnealingLR, which starts at the full learning rate with zero warmup. With batch size 4 and noisy gradients early in training, this caused large loss spikes in epoch 1, triggering the

patience-3 early stopping callback before any meaningful learning occurred. Selective retraining reproduced the same stopping epoch deterministically, confirming this was a learning rate dynamics issue, not hardware.

Cell 10_GPU uses OneCycleLR with a 10% linear warmup phase: the learning rate starts at $\text{max_lr} / 10$, ramps up linearly for the first 10% of training steps, then follows a cosine decay to $\text{max_lr} / 10,000$. With batch size 16 (smoother gradient estimates) and patience 5, all five folds should run to completion for the first time.

7. GPU Lab — Full Ablation Study

The ablation study systematically removes or replaces one component at a time from the full Neuro-DT architecture. Every variant uses the identical Fold 4 data split, scaler, loss function, and evaluation metric, so performance differences are directly attributable to the changed component.

ID	Model name	MRI	Tabular	Fusion	What it proves
A0	Neuro-DT Full	✓	✓	Transformer 2L	Reference — already done. Loaded from existing checkpoint.
A1	Tabular-Only MLP	✗	✓	MLP	MRI contribution = 0.912 minus A1 AUC. Most important comparison.
A2	CNN Only	✓	✗	Linear	Tabular contribution = A0 minus A2. Also shows CNN-only vs literature.
A3	CNN + MLP Fusion	✓	✓	MLP	Transformer value: does attention outperform standard nonlinear fusion?
A4	CNN + Linear	✓	✓	Linear	Minimum multimodal baseline. A3 minus A4 = MLP fusion contribution.
A5	Transformer 1 Layer	✓	✓	Transformer 1L	Is depth of 2 Transformer layers justified over 1 layer?
A6	No Class Weights	✓	✓	Transformer 2L	Impact of weighted loss on MCI recall (the hardest class).
B1	Logistic Regression	✗	✓	Linear	Classical ML floor — tabular only, linear decision boundary.
B2	SVM (RBF kernel)	✗	✓	Kernel	Strong classical tabular baseline with nonlinear kernel.
B3	Random Forest	✗	✓	Ensemble	Nonlinear ensemble baseline, handles feature interactions.
B4	Gradient Boosting	✗	✓	Ensemble	Best-known classical tabular method — upper bound for no-MRI.

Resuming after kernel death (Cell A2): Cells A2 and A3 both read `ablation_results.json` at startup and skip any variant already saved there. If the kernel dies mid-run, run Cell A5 to see what completed, then re-run Cell A2 — it resumes exactly where it stopped. Cell A4 works with partial results and shows GPU-pending placeholders for any missing variant.

8. Comparing CPU vs GPU Results in Your Thesis

Notebook	What it contributes to the thesis
CPU Workhorse	Confirms AUC 0.912 is reproducible (A0 re-evaluation on the same Fold 4 split). Proves 3D MRI is essential: delta AUC = 0.912 minus A1 result. Proves deep learning outperforms all classical tabular-only methods (B1–B4). Generates all Digital Twin simulation figures (Cells 13–16). Provides a partial ablation table with GPU placeholders for supervisor review.
GPU Lab	Complete 5-fold cross-validated AUC with mean and standard deviation — statistically stronger than single-fold. Full ablation study (A0–A6, B1–B4) justifying every architectural decision with evidence. Full SHAP analysis on both imaging and tabular branches (CPU OOM prevented imaging SHAP previously). Potential AUC improvement from more stable training with OneCycleLR + batch 16.

How to write up the methodology comparison

"Initial training was performed on an Azure Standard_E4ds_v4 CPU-only compute instance using 5-fold stratified cross-validation on 1,549 ADNI scans. Due to training instability caused by CosineAnnealingLR without warmup at batch size 4, four of five folds experienced premature early stopping at epochs 1–3. Fold 4 ran to completion (15 epochs, AUC = 0.912). Full 5-fold cross-validation was subsequently completed on a GPU-equipped university lab machine using OneCycleLR with 10% linear warmup, batch size 16, and mixed-precision training, yielding AUC = [GPU_MEAN] ± [GPU_STD] across five folds."

How to write up the ablation findings

"Table X presents the ablation study evaluated on the Fold 4 validation set (n=310). Removing the 3D MRI branch (A1) reduces macro AUC from 0.912 to [A1_AUC], a decrease of [DELTA] percentage points, demonstrating that structural neuroimaging is the primary driver of diagnostic performance. Replacing the Transformer encoder with MLP fusion (A3) reduces AUC to [A3_AUC], confirming that cross-modal attention outperforms standard nonlinear fusion. The best classical ML baseline operating on tabular features alone achieves AUC = [MAX_B], establishing that the 3D imaging pipeline contributes [0.912 minus MAX_B] AUC points above what any tabular-only approach can achieve."